



Structure-Preserving Residual Initialization for Dense-to-MoE Upcycling

Visal Soeun¹, Jaehyun Sim^{1,2}, Kevin Min¹

¹Department of Electrical Engineering & Computer Science, Massachusetts Institute of Technology, Cambridge, MA 02139, USA.

²System Design & Management, Massachusetts Institute of Technology, Cambridge, MA 02139, USA.

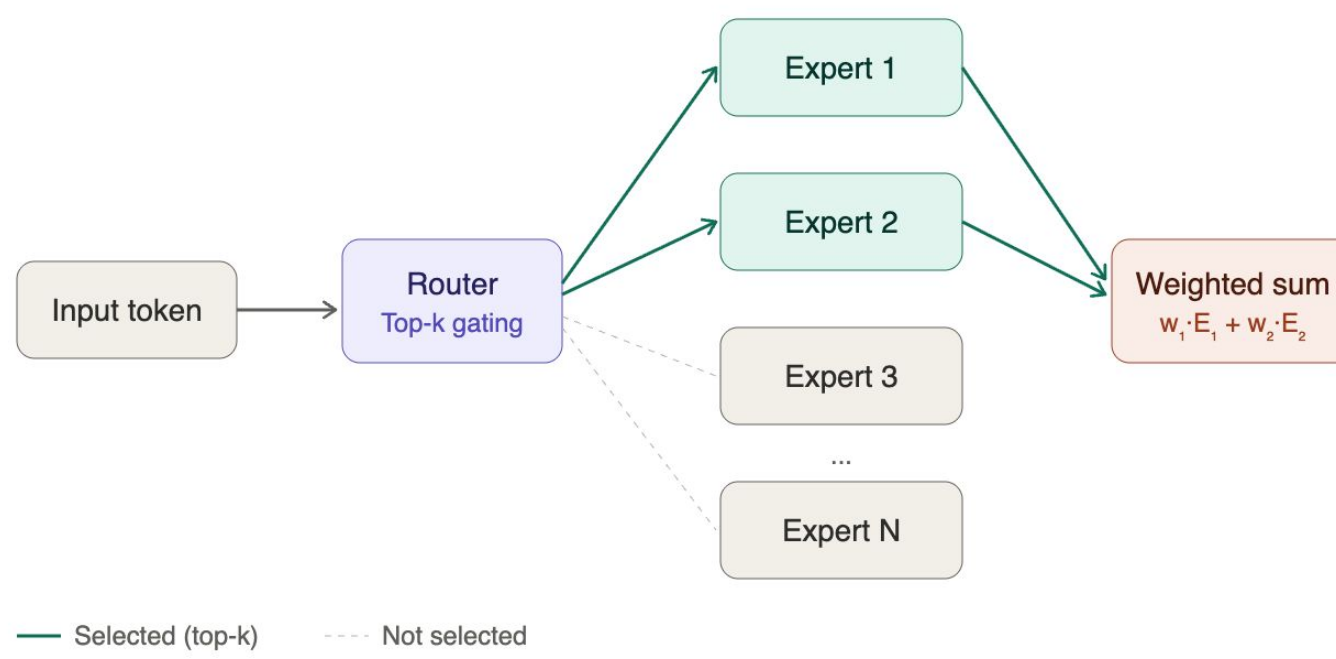


Mixture-of-Expert Models

Scale model capacity without scaling compute

Mixture-of-Expert (MoE) models scale model capacity while keeping per-token compute low by routing each token to a **subset of expert networks** via a learned router.

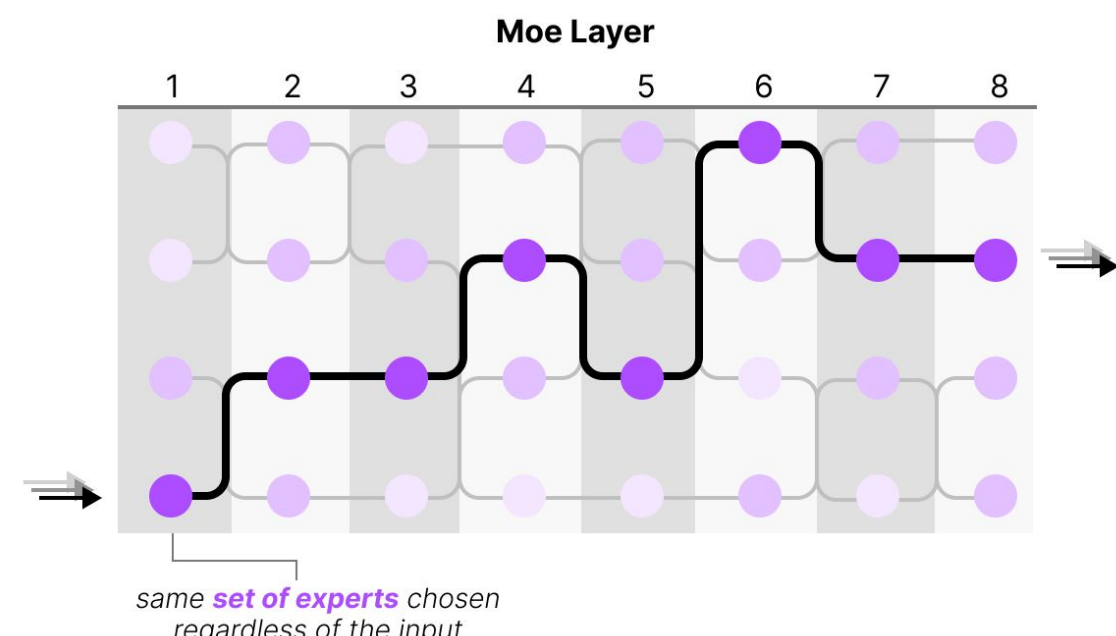
- Each transformer block replaces the single FFN with **N parallel expert FFNs** and a gating network.
- The router selects **top-k experts** per token (typically k=2), combining their outputs via **weighted sum**.
- MoE achieves the **parameter count of a large model with the FLOPs of a small one** (e.g., Qwen1.5-MoE-A2.7B: 14B total params, only 2.7B active per token).



Model	Total Params	Active Params	Experts	Top-k
Mixtral 8x7B	46.7B	12.9B	8	2
DeepSeek-V3	671B	37B	256	8
Qwen3-235B	235B	22B	128	8

Challenges of Mixture-of-Expert Models

- Training cost:** Training MoE from scratch is **prohibitively expensive**. Converting pretrained dense models is far more practical.
- Load balancing:** If the router favors a small subset of experts, **most experts go unused**. Auxiliary balancing losses or load-aware routing are needed.
- Expert redundancy:** Naively initialized experts can remain **near-identical** throughout training, failing to specialize and wasting capacity.



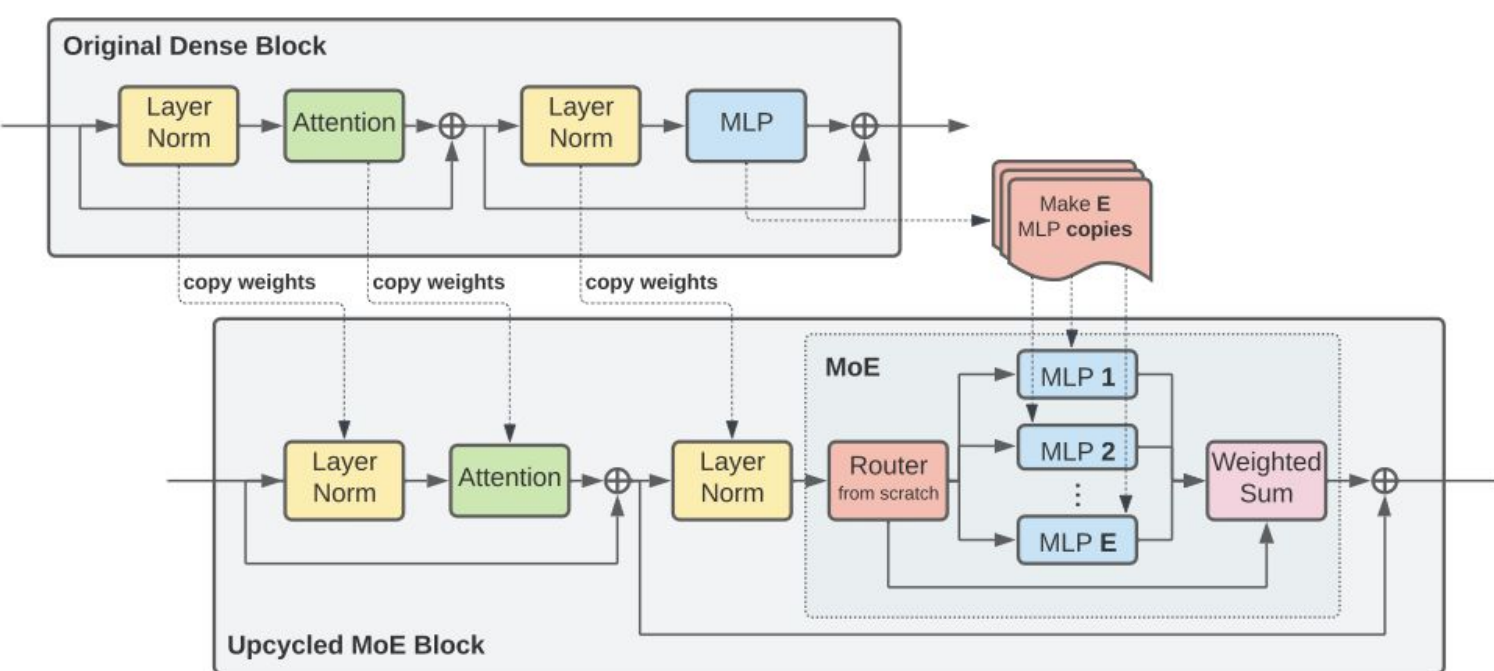
Graphic from <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts>

same set of experts chosen regardless of the input

Dense-to-MoE Upcycling

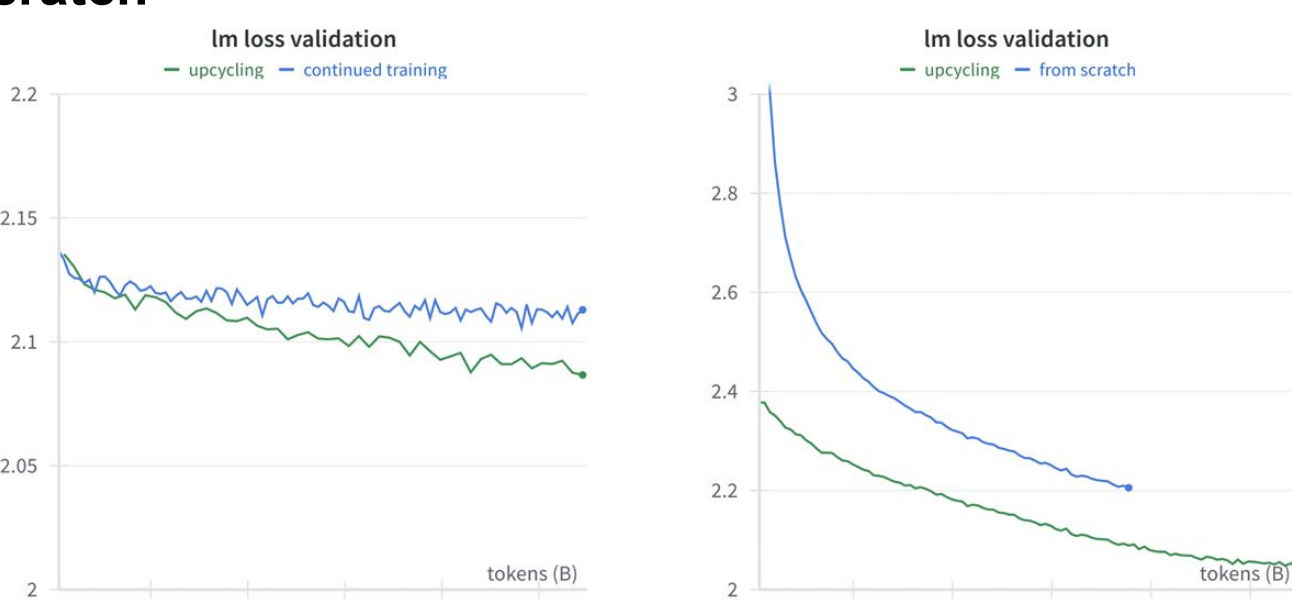
Convert a pre-trained dense model into MoE without training from scratch

- In a standard transformer, each layer has one MLP block that processes every token. In MoE, **this single MLP is replaced with N copies (experts)**, and a router learns to select the best k experts for each token.
- Upcycling initializes these experts from the **pretrained MLP weights** rather than from random initialization, preserving the knowledge the model has already learned.
- Attention layers, normalization layers, and embeddings are copied directly.



Upcycling is more efficient than training MoE from scratch

- Reuses pretrained knowledge:** Learned representations are kept intact. Only the experts need initialization.
- Lower cost:** Training MoE from scratch requires the full pretraining budget. Upcycling achieves **competitive performance with a fraction of the compute**.
- Faster convergence:** Experts start from a **strong initialization** rather than random weights, reaching competitive performance in **fewer training steps**.



Existing initialization strategies face a fundamental trade-off

Direct copy: Each expert receives identical weights from the dense FFN. Preserves pretrained performance, but experts fail to differentiate (symmetry problem).

$$W_e := W_{FFN}$$
$$\text{s.t. } W_e^i = W_{FFN}[i:d : (i+1) \cdot d]$$

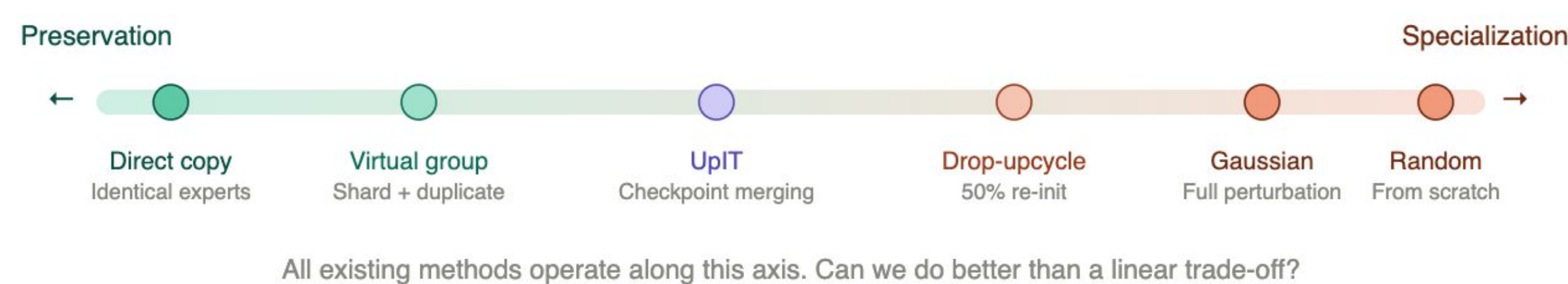
Gaussian perturbation: Add i.i.d noise scaled by weight magnitude. Breaks symmetry, but perturbs the full weight matrix, risks destroying learned structure.

$$W_e := W_{FFN} + \mathcal{N}(0, I) \cdot \sigma \cdot \text{avg}(|W_{FFN}|)$$

Virtual group (NVIDIA): Shard the MLP along the intermediate dimension, then duplicate each shard. Ensures MoE output matches dense model at initialization, but experts within each group start identical.

UpIT: Use intermediate checkpoints saved during instruction tuning as different experts, then merge via genetic algorithm. Experts are naturally diverse, but requires multiple training runs to produce checkpoints.

Drop-upcycle: Copy weights into experts, then randomly re-initialize 50% of each expert's parameters. Forces diversity, but which 50% to drop is random.



Structure-Preserving Residual Initialization

Not all directions in the weight space are equally important

Idea: A pretrained weight matrix contains dominant structure (learned representations) and a residual component (less significant directions). Expert diversity should come from the **residual**, not the structure.

Structure-Preserving Residual Initialization

Step 1: Decompose each MLP weight matrix via SVD

$$W_{FFN} = U \Sigma V^T$$

Split the singular values into two components:

$$\Sigma = \Sigma_{\text{struct}} + \Sigma_{\text{res}}$$

where Σ_{struct} corresponds to the top-k largest singular values

(dominant, learned structure)

and $\Sigma_{\text{res}} = \Sigma - \Sigma_{\text{struct}}$ (remaining, less significant directions).

Step 2: Sample new residual singular values per expert

For each expert i, sample a new residual:

$$(\tilde{\Sigma}_{\text{res},i})_{mm} \sim \mathcal{N}((\Sigma_{\text{res}})_{mm}, \frac{\sigma^2}{m})$$

Higher variance is allocated to smaller singular values.

Diversity is concentrated where it matters least.

Larger singular values receive minimal perturbation.

Dominant structure is preserved.

Step 3: Reconstruct per-expert weight matrices

$$W_i = U \cdot [\Sigma_{\text{struct}} + \tilde{\Sigma}_{\text{res},i}] \cdot V^T$$

U, V^T, and Σ_{struct} are shared across all experts.

Only $\tilde{\Sigma}_{\text{res}}$ differs per expert.

SVD decomposition is a one-time cost per layer (negligible overhead).

$$W_{FFN} = U \Sigma V^T$$

s.t. $\Sigma = \Sigma_{\text{struct}} + \Sigma_{\text{res}}$

where Σ_{struct} corresponds to top k largest singular values and $\Sigma_{\text{res}} = \Sigma - \Sigma_{\text{struct}}$

$$W = U \Sigma V^T$$

Split the singular values into two components:

$$\Sigma = \Sigma_{\text{struct}} + \Sigma_{\text{res}}$$

where Σ_{struct} corresponds to the top-k largest singular values

(dominant, learned structure)

and $\Sigma_{\text{res}} = \Sigma - \Sigma_{\text{struct}}$ (remaining, less significant directions).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

SVD decomposition is a one-time cost per layer (negligible overhead).

Implementation

Implementation Logic: Initialization

```
def initialize_svd(dense_model, moe_model, svd_dimension, svd_scale):
    for (dense_layer, experts) in (dense_model, moe_model):
        weights = dense_layer.weight
        experts.fill(0)

        # Copy svd perturbation from dense model to experts
        for expert in experts:
            # Get the row indices in weights corresponding to this expert
            row_start, row_end = get_row_indices(expert)
            expert_gate[row_start : row_end] = svd_perturb(weights_gate[row_start : row_end], svd_dimension, svd_scale)
            expert_up_proj[row_start : row_end] = svd_perturb(weights_up_proj[row_start : row_end], svd_dimension, svd_scale)
            expert_down_proj[row_start : row_end] = svd_perturb(weights_down_proj[row_start : row_end], svd_dimension, svd_scale)
```

Setup for Upcycling

Component	Dense (Qwen1.5-1.8B)	MoE (Qwen1.5-MoE-A2.7B)	Match
hidden_size	2048	2048	exact
layers	24	24	exact
attention heads	16	16	exact
KV heads	16	16	exact
vocab_size	151936	151936	exact
FFN intermediate	5504	—	—
expert intermediate	—	1408	—
shared expert intermediate	—	5504 (overridden)	—
num experts	—	60, top-4 active	—

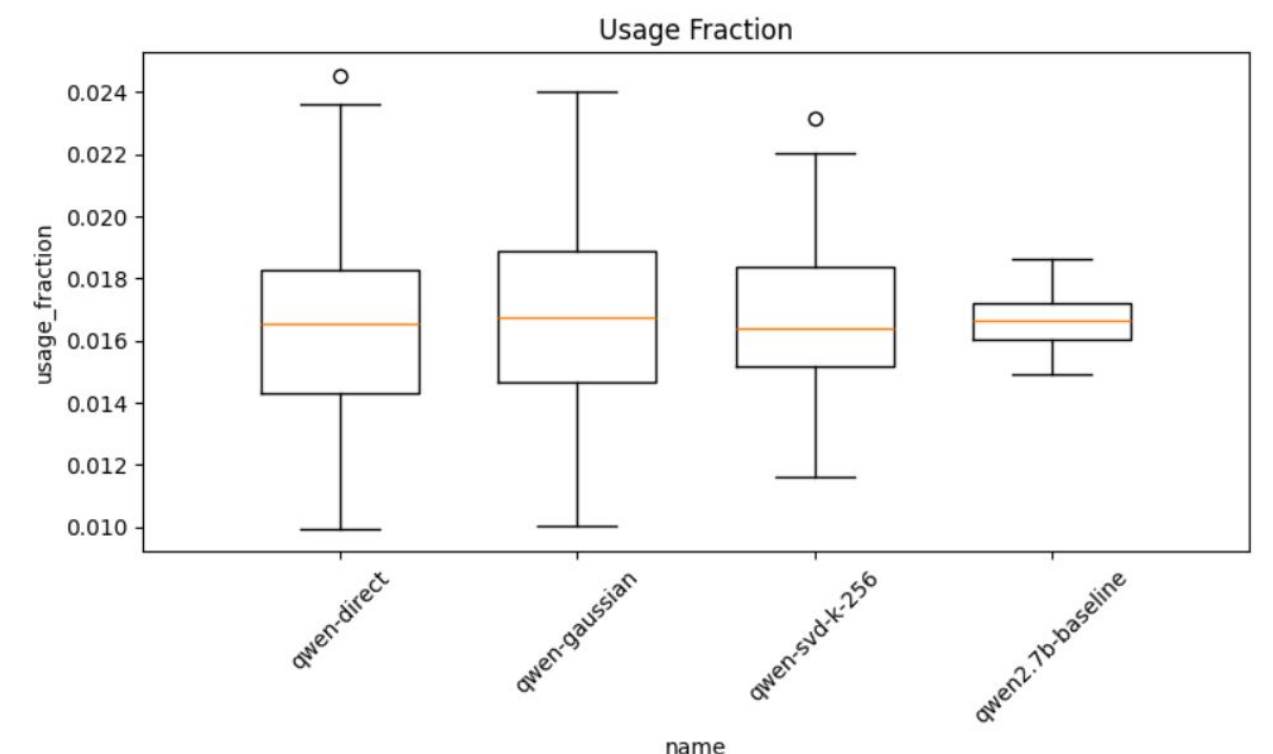
Additional details

- Used cosine annealing w/ warmup (80 warmup steps, 400 total)
 - Batch size of 8
- ~5e-4 LR seemed ideal - high enough to yield significant divergence in models, without overshooting gradient descent
- Used 0.05-0.2 sigma for both Gaussian noise and SVD noise
 - Literature seems to suggestion lower, but with our limited compute any lower had very little impact

Results & Benchmarks

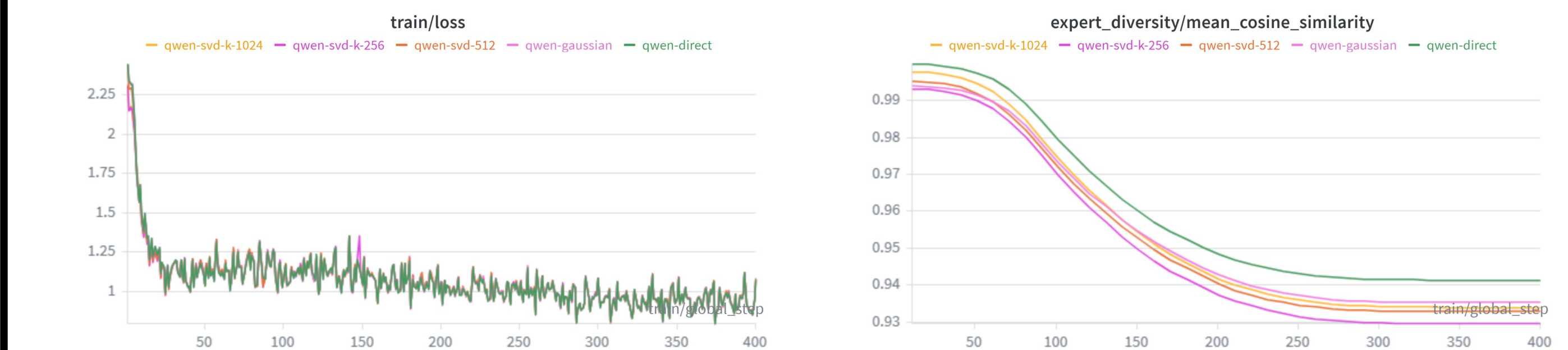
Training losses near identical for various initializations

- Relatively small training size:** Limited compute budget (small training size) makes it difficult to observe meaningful loss separation between methods.
- Stable baseline model:** The dense Qwen1.5 baseline is already well-trained, with loss converged to a stable minimum. The presence of shared experts may further reduce the impact of expert initialization.



SVD initialization produces more diversified experts

- SVD leads to lower expert similarity:** Experts initialized with SVD show lower pairwise cosine similarity than both direct copy and Gaussian perturbation, even at rank k=1024 where initial similarity is higher than Gaussian noise.
- More even usage rates:** Token routing is more evenly distributed under SVD initialization, with lower interquartile range (IQR) in expert usage rates.
- Low sample size:** Further testing at larger scale is needed to determine whether these diversification gains translate to measurable performance improvements.



Benchmark: GSM8K (Few-shot Prompt)

We evaluate all models on GSM8K math reasoning after 400 training steps using 5-shot prompting. The dense Qwen1.5-0.5B baseline is included as reference.

- Training was limited to 400 steps due to compute constraints.
- SVD initialization (k=512, sigma=0.5) achieves the **highest GSM8K accuracy** among all upcycled models, **outperforming both direct copy and Gaussian perturbation**.
- All upcycled models show a slight accuracy drop compared to the dense baseline, consistent with expected behavior after only 400 steps of MoE fine-tuning.
- Preliminary testing on Mistral 7B → Mixtral 8x7B (300 steps) shows SVD initialization **achieving 1.2% accuracy gain over Gaussian perturbation**.
- More thorough training runs with larger and modern model architectures, longer training horizons, and comprehensive benchmark suites are suggested to validate these findings.

Methods	σ	GSM8K Accuracy
Direct Copy	-	447/1319
Gaussian	0.5	336/1319
Gaussian	1.0	336/1319
SVD k=512	0.5	368/1319
SVD k=512	1.0	357/1319

Conclusion

- Structure-preserving residual initialization **achieves comparable training loss to both direct copy and Gaussian perturbation baselines** on Qwen1.5-0.5B → Qwen1.5-MoE-A2.7B.
- The method did not produce faster convergence over baselines at this scale and compute budget.
- SVD-initialized models achieve the **highest GSM8K accuracy** among all initialization methods after 400 training steps.
- Preliminary results on Mixtral scale (300 steps) show a 1.2% accuracy gain over Gaussian perturbation, **suggesting the method's advantages become more pronounced at larger scale**.
- Experts initialized with our method show lower pairwise cosine similarity than direct copy, indicating **greater diversification** of experts from initialization.
- Token routing is **more evenly distributed across experts**, suggesting the router utilizes all experts rather than collapsing to a subset.
- More thorough training runs with larger and modern model architectures are suggested to validate these findings.

Contribution

Dense to Mixture-of-Expert upcycling idea was suggested and developed by VS and KM. SVD-based initialization method was proposed and initially implemented by JS with support of KM and VS. Model upcycling pipeline and training infrastructure was built by JS. Benchmarking and evaluation was done by VS and KM. Hyperparameter tuning and ablation studies were conducted by VS. Expert diversification and load balancing analysis was performed by KM. Poster and presentation materials were prepared by JS, KM, and VS.

References

- [1] Horoi et al., **Less is More: Undertraining Experts Improves Model Upcycling**, arXiv, 2025.
- [2] Hui et al., **Upcycling Instruction Tuning from Dense to Mixture-of-Experts via Parameter Merging**, ACL, 2025.
- [3] Komatsuzaki et al., **Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints**, arXiv, 2023.
- [4] Liew et al., **Scaling Laws for Upcycling Mixture-of-Experts Language Models**, arXiv, 2025.